

DOCUMENT: ForeShift Backend Service — Build Specification

FOR: Backend Developer | **FROM:** ForeShift LLC | **Confidential — under NDA**

ForeShift Demand Service

Backend specification for the zone-demand + event/weather calculation service.

What this service is. A standalone API that computes **zone demand** for a given zone, concept, day, and daypart — starting from a fixed base score, then applying live event and weather adjustments. The front-end app (currently Bubble) calls this service and displays what it returns. The service performs all demand math; the app only orchestrates and displays.

Two hard requirements up front (details in §5). (1) This must be a **standalone, platform-agnostic REST API** — not coupled to Bubble — so the front-end can change or an external API can reuse it later. (2) The tuning coefficients must be **editable by a non-technical admin (the owner) without code changes or redeloys** — via a simple admin interface — because they are recalibrated regularly.

1 · What the Service Does (Overview)

On each request, the service:

1. Receives a **zone, concept, day, daypart**, plus **today's nearby events** and **weather** (supplied by the caller).
2. Looks up the fixed **base zone-demand score** for that zone × concept × day × daypart (from a stored table ForeShift provides).
3. Applies **event lift** (additive) and **weather adjustment** (multiplicative), then caps the result.
4. Converts the final score to a **demand band** and returns it.

Terminology: this service outputs **zone demand** (demand for a concept in a zone), never "restaurant demand." Keep this wording in any field names, logs, or responses.

1.1 · Fixed reference values (dayparts, zones, concepts)

Four dayparts (the `daypart` field accepts these keys; clock windows shown for reference):

Daypart key	Clock window
<code>morning</code>	6:00 AM – 11:00 AM
<code>midday</code>	11:00 AM – 4:00 PM
<code>dinner</code>	4:00 PM – 9:00 PM
<code>late</code>	9:00 PM – 2:00 AM

13 zones (exact strings — must match the base-score file and incoming requests): Woodward Core; Downtown Detroit (Core); Foxtown / Stadium District; Greektown / Casino District; Financial District; Midtown; Corktown; Eastern Market; New Center / North End; Riverfront / RiverWalk; Mexicantown; Core City / Woodbridge; Southwest Detroit.

9 concepts (exact strings): Fine Dining; Upscale Casual; Casual Dining; Fast Casual; Coffee Shop; Breakfast / Brunch Cafe; Sports Bar; Cocktail Lounge; Neighborhood / Casual Bar.

These strings are the source of truth for joins/lookups. The base-score file (§4.1) uses these exact names; requests must too. A mismatch (e.g., "Woodward Corridor" vs. "Woodward Core") silently breaks the lookup.

2 · The Calculation

The core formula (this is the exact model logic):

```
# Final zone-demand score
final = MIN( (base_score + event_lift) * weather_factor , 150 )

# where
event_lift    = event_magnitude * concept_event_affinity * proximity
weather_factor = 1 - (weather_severity * concept_weather_affinity)
```

Notes on the logic (do not change these behaviors):

- **Events are additive** — an event injects outside crowds independent of the baseline (a big event on a slow day must still register).
- **Weather is multiplicative** — it scales whatever demand exists up or down.
- **If multiple events** are nearby, sum their individual lifts before adding to base.
- **Cap** the final score at 150.
- **Then re-band** the capped score (bands in §4).

Worked example (build test - uses the DUMMY values from Section 4, so your build should match this exactly). Base score 30.0, one major event (dummy magnitude 50) at full proximity 1.0, dummy event-affinity 0.50, storm weather (severity 0.5), dummy weather-affinity 0.35.
 $event_lift = 50 \times 0.50 \times 1.0 = 25 \rightarrow \text{after event} = 55.0 \rightarrow weather_factor = 1 - (0.5 \times 0.35) = 0.825 \rightarrow 55.0 \times 0.825 = \mathbf{45.38} \rightarrow \text{band} = \text{MODERATE}$. Built with the Section 4 dummy values, your service should produce **45.38 / Moderate** for these inputs. (Once real coefficients are loaded, the same inputs will yield a different number - that is expected.)

3 · Inputs & Outputs (API Contract)

SAFE TO SHARE FULLY

Request (what the caller sends)

```
POST /zone-demand
{
  "zone": "Woodward Core",
  "concept": "Sports Bar",
  "day": "Sat",
  "daypart": "dinner",           // morning | midday | dinner | late
  "events": [
    { "class": "Major stadium game", "proximity": 1.0 }
  ],
  "weather_severity": 0.5        // 0 normal, 0.5 storm, -0.2 ideal-day boost
}
```

Response (what the service returns)

```
{
  "zone": "Woodward Core",
  "concept": "Sports Bar",
  "base_score": 30.0,
  "final_score": 45.38,
  "band": "Moderate",
  "event_applied": true
}
```

The caller (Bubble) is responsible for fetching events (e.g., Ticketmaster) and weather, and for computing each event's **proximity** (0–1 based on distance to the zone). The service just consumes those values. If you prefer the service to fetch events/weather itself, flag it — either split works, but keeping fetch in the caller keeps this service pure math.

4 · Reference Data (Tables the Service Reads)

The service reads four stored tables. **Three are ForeShift proprietary values** — for building, use the DUMMY values below; ForeShift will load the real values into the admin config (\$5).

4.1 · Base zone-demand scores **PROVIDED BY FORESHIFT**

The fixed base score per zone × concept × day × daypart. ForeShift supplies this as a data file (~3,276 rows) to load into the service's datastore. Structure: `zone, concept, day, daypart, base_score`. Look up the row matching the request.

4.2 · Event affinity by concept **DUMMY — owner loads real**

How strongly each concept captures event crowds (0–1). **Build with these placeholder values**; real values loaded by owner via admin:

Concept	Event Affinity (DUMMY)
Sports Bar	0.50
Cocktail Lounge	0.50
Neighborhood / Casual Bar	0.50
Fast Casual	0.50
Casual Dining	0.50
Upscale Casual	0.50
Coffee Shop	0.50
Breakfast / Brunch Cafe	0.50
Fine Dining	0.50

(All set to 0.50 as neutral placeholders. Real values differ per concept and are owner-controlled.)

4.3 · Event magnitude by class **DUMMY — owner loads real**

Event class	Magnitude (DUMMY)
Major stadium game	50
Concert / large show	30
Festival day	20
Minor event	10

4.4 · Weather affinity by concept **DUMMY — owner loads real**

Used in `weather_factor = 1 - (severity × weather_affinity)` . Placeholder 0.35 for all; real values owner-controlled.

4.5 · Band thresholds **SAFE TO SHARE**

Band	Score range
Exceptional	110–150
Peak	85–109
High	65–84
Moderate	40–64
Light	20–39
Minimal	0–19

5 · The Two Hard Requirements

5.1 · Standalone, platform-agnostic REST API

Requirement: Build as an independent REST API (JSON in/out) that any client can call. Do NOT couple it to Bubble internals or build it as a Bubble plugin. It must run and be testable on its own (e.g., callable via curl/Postman) with no dependency on the front-end.

Reasons (for context): (1) the front-end may migrate off Bubble later and must reuse this service unchanged; (2) this same service is intended to later power an external partner API. Both require it to be front-end-agnostic. Recommended: a serverless function (AWS Lambda / Google Cloud Function / similar), which is low-cost and auto-scaling.

5.2 · Owner-editable coefficients (no code, no redeploy)

Requirement: The coefficients in §4.2, §4.3, §4.4 must be editable by a **non-technical admin (the owner)** through a **simple admin interface** — change a value, save, and the service uses the new value on the next request. Updating these must NOT require editing code, redeploying, or developer involvement.

Implementation (developer's choice, but must meet the above):

- Store coefficients in a **database table or config store**, NOT hardcoded in the service code.
- Provide a **simple admin screen** (a basic form listing each coefficient as an editable field with a Save button) that the owner can access. This can be a lightweight standalone admin page, or — if cleaner for this stack — an admin-only table in the existing app that the owner edits and the service reads.
- The service reads current coefficient values at request time (or caches with a short refresh), so edits take effect without redeploy.

Why this matters (context): ForeShift recalibrates these coefficients regularly against real venue results. The owner needs to adjust values frequently and independently. A build that hardcodes them (requiring a developer each change) does not meet the requirement.

6 · Recalibration Flow (Build For This)

The service must support this ongoing loop:

1. Venues report actual busy-ness (captured in the app).
2. ForeShift compares predicted zone demand vs. reported actuals (analysis done by owner, outside the service).
3. Owner updates coefficient values via the admin interface (§5.2).

4. Service immediately uses updated values — all future forecasts improve.

You do not need to build the analysis (step 2) — just ensure steps 3–4 work: coefficients are owner-editable and take effect live. Optionally, log each request's inputs/outputs to support later analysis.

7 · What ForeShift Will Provide

Item	When
Base zone-demand score file (§4.1, ~3,276 rows)	At build start (real data — under NDA)
Real coefficient values (§4.2–4.4)	Loaded by owner via admin after build — NOT needed to build
The 13 zone names, 9 concept names, daypart windows, band thresholds	In this spec

8 · Summary Checklist for the Developer

- ☐ Standalone REST API, JSON in/out, callable independently (not Bubble-coupled)
- ☐ Implements the formula in §2 exactly (additive events, multiplicative weather, cap 150, re-band)
- ☐ Passes the Section 2 worked example (dummy-value inputs → 45.38 → Moderate)
- ☐ Reads base scores from the provided data file (§4.1)
- ☐ Coefficients in an editable store, NOT hardcoded (§5.2)
- ☐ Simple admin interface for the owner to edit coefficients (no code/redeploy)
- ☐ Coefficient edits take effect live (next request)
- ☐ Outputs "zone demand" terminology throughout
- ☐ Deployable as serverless (low-cost, auto-scaling)
- ☐ (Optional) Request logging to support recalibration analysis

Confidential — ForeShift LLC — provided under NDA — not for distribution